Fatec
Diadema
Luigi Papaiz

Programação para Dispositivos Móveis I

Prof. Marcos Vasconcelos de Oliveira

Marcos.vasconcelos4@fatec.sp.gov.br

Fatec
Diadema
Luigi Papaiz

Aula 4 – Estrutura Condicional e Estrutura de Repetição – Linguagem Kotlin

Prof. Marcos Vasconcelos de Oliveira

Marcos.vasconcelos4@fatec.sp.gov.br

Site para rodar os programas

- https://repl.it/repls/ShockedWeightyComputergame#main.kt

O Mapa da Programação: As 5 Fases

1. Entender o Problema (Objetivo)

2. Detalhar o Problema (**E/P/S**)
   Entradas, Processamento, Saídas

3. Fluxograma

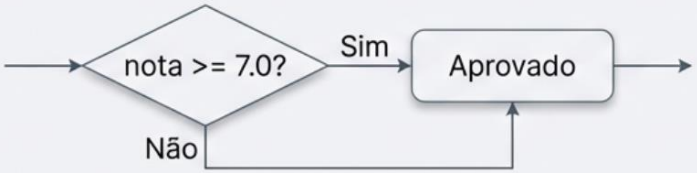
4. Teste de Mesa

5. Codificação (**Kotlin**)

Dúvidas? Fases para programação

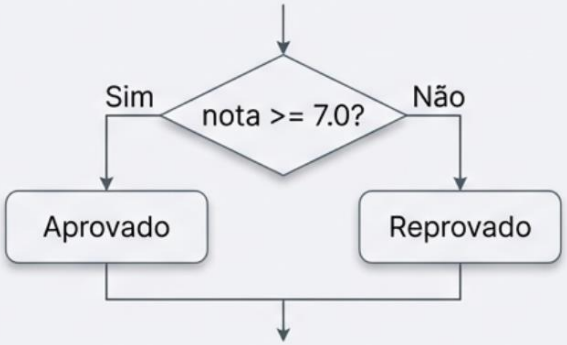
- 1ª fase – Entender o Problema
- 2a. Fase - Detalhar o Problema
  Passo a Passo –
- E/P/S
  E ==>
- P ==>
- S ==>
- 3a. Fase = Fluxograma/ Diagrama de Blocos
- 4a. Fase = Testes / Teste de Mesa
- 5a. Fase = Codificação = Português Estruturado/Linguagem

Condicional Simples




```
val nota: Double = 8.3
if (nota >= 7.0) println('Aprovado')
```

Condicional Composta




```
val nota: Double = 5.3
if (nota >= 7.0) {
    println('Aprovado')
} else {
    println('Reprovado')
}
```

Condicional Simples

- Condicional

```
fun main() {
    val nota: Double = 8.3

    if (nota >=7.0)
    println("Aprovado")
}
```

Fatec
Diadema
Luigi Papaiz

Condicional Simples – Código Fonte

```
1. fun main() {
2.     val nota: Double = 8.3

3.     if (nota >=7.0)
4.     println("Aprovado")
5. }
```

Fatec
Diadema
Luigi Papaiz

Condicional Composto

```
fun main() {
   val nota: Double = 5.3

   if (nota >=7.0) {
      println("Aprovado")
   } else
   {
      println("Reprovado")
   }
}
```

Condicional Composto – Código Fonte

```
1. fun main() {
2.     val nota: Double = 5.3

3.     if (nota >=7.0) {
4.         println("Aprovado")
5.     } else
6.     {
7.         println("Reprovado")
8.     }
9. }
```

Fatec Diadema Luigi Papaiz

O Superpoder do Kotlin: If como Expressão

Atribuição direta de valores baseada em condições.

```
val nota1: Int = 10
val nota2: Int = 5

val maiorNota = if (nota1 > nota2) {
    println('Processando...')
    nota1 // Return Value
} else {
    println('Processando...')
    nota2 // Return Value
}
println('A maior nota é \$maiorNota')
```

Condicional

```
fun main() {
    val nota1: Int = 10
    val nota2: Int = 5

    val maiorNota = if (nota1 > nota2) {
        println("Processando...")
        nota1 //ultima expressão que será retornada
    } else
    {
        println("Processando...")
        nota2 //ultima expressão que será retornada
    }
    println("A maior nota é $maiorNota")
}
```

Condicional – Código Fonte

```
fun main() {
    val nota1: Int = 10
    val nota2: Int = 5
    val maiorNota = if (nota1 > nota2) {
        println("Processando...")
        nota1
    } else
    {
        println("Processando...")
        nota2
}
    println("A maior nota é $maiorNota")
}
```

Fatec
Diadema
Luigi Papaiz

Condicional

```
fun main() {
    val nota1: Int = 10
    val nota2: Int = 5

    val maiorNota = if (nota1 > nota2) nota1 else nota2
    println("A maior nota é $maiorNota")
}
```

Fatec
Diadema
Luigi Papaiz

Condicional – Código Fonte

```
fun main() {
    val nota1: Int = 10
    val nota2: Int = 5

    val maiorNota = if (nota1 > nota2) nota1 else nota2
    println("A maior nota é $maiorNota")
}
```

Fatec
Diadema
Luigi Papaiz

Operador range

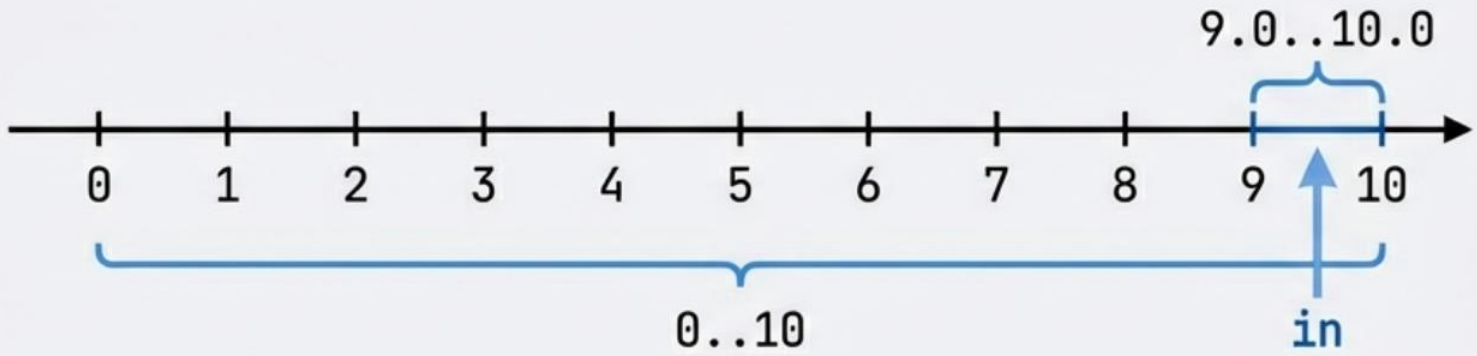
```
fun main() {
    val nota: Double = 8.50

    if (nota in 9.0..10.0) {
        println("Nota excelente")
    } else if (nota in 6.0..8.9) {
        println("Aprovado")
    } else if (nota in 4.0..5.9) {
        println("Reavaliação")
    } else if (nota in 0.0..3.9) {
        println("Reprovado")
    } else {
        println("Nota Inválida")
    }
}
```

- Operador Range

Fatec
Diadema
Luigi Papaiz

O Operador Range (Intervalos)



O Problema: Complexidade com If/Else

Código Verboso

```
if (nota in 9.0..10.0) {
    println('Nota excelente')
} else if (nota in 6.0..8.9) { println('Aprovado')
} else if (nota in 4.0..5.9) { println('Reavaliação')
} else if (nota in 0.0..3.9) { println('Reprovado')
} else {
    println('Nota Inválida')
}
```

Operador range – Código Fonte

```
fun main() {
        val nota: Double = 8.50
        if (nota in 9.0..10.0) {
                println("Nota excelente")
        } else if (nota in 6.0..8.9) {
                println("Aprovado")
        } else if (nota in 4.0..5.9) {
                println("Reavaliação")
        } else if (nota in 0.0..3.9) {
                println("Reprovado")
        } else {
                println("Nota Inválida")
        }
}
```

Fatec
Diadema
Luigi Papaiz

Estrutura When

```
1  fun main() {
2      val nota: Double = 8.9
3      when (nota) {
4      in 9.0..10.0 -> println("Nota excelente")
5      in 6.0..8.9 -> println("Aprovado")
6      in 4.0..5.9 -> println("Reavaliação")
7      in 0.0..3.9 -> println("Reprovado")
8      else -> println("Nota Inválida")
9      }
10 }
```

```
val nota: Double = 8.9
when (nota) {
    in 9.0..10.0 -> println('Nota excelente')
    in 6.0..8.9  -> println('Aprovado')
    in 4.0..5.9  -> println('Reavaliação')
    in 0.0..3.9  -> println('Reprovado')
    else         -> println('Nota Inválida')
}
```

Mapeamento Direto

Mapeamento Direto

Mapeamento Direto

Mapeamento Direto

Mapeamento Direto

Estrutura When – Código Fonte

```
fun main() {
        val nota: Double = 8.9
        when (nota) {
                in 9.0..10.0 -> println("Nota excelente")
                in 6.0..8.9 -> println("Aprovado")
                in 4.0..5.9 -> println("Reavaliação")
                in 0.0..3.9 -> println("Reprovado")
                else -> println("Nota Inválida")
        }
}
```

Fatec
Diadema
Luigi Papaiz

While

```
fun main() {
   var opcao: Int = 0
   println("Digite uma opção ")
   while (opcao != -1) {
      opcao = readLine().toString().toInt()
      println(opcao is Int)
      println("Opção escolhida $opcao")
   }
   println("Fim do Processamento")
}
```

While – Código Fonte

```
1.  fun main() {
2.      var opcao: Int = 0
3.      println("Digite uma opção ")
4.      while (opcao != -1) {
5.          opcao = readLine().toString().toInt()
6.          println(opcao is Int)
7.          println("Opção escolhida $opcao")
8.      }
9.      println("Fim do Processamento")
10. }
```

Fatec Diadema Luigi Papaiz

While

```
fun main() {
    var contador: Int = 0
    while (contador <= 10) {
        println(contador)
        contador++
    }
    println("Fim do Processamento")
}
```

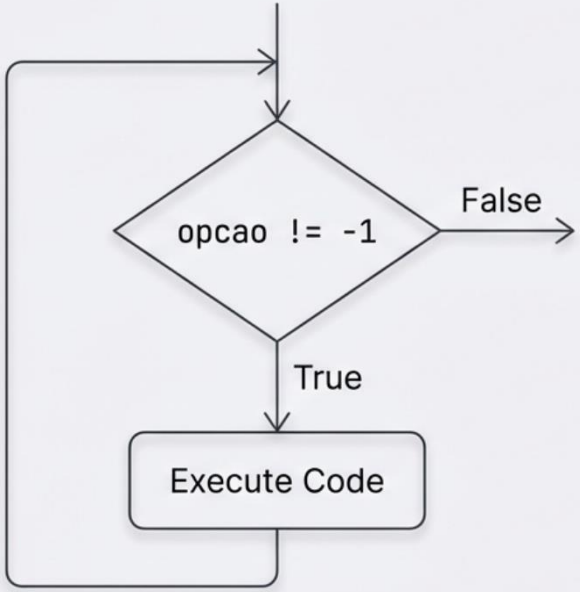
While – Código Fonte

```
1. fun main() {
2.     var contador: Int = 0
3.     while (contador <= 10) {
4.         println(contador)
5.         contador++
6.     }
7.     println("Fim do Processamento")
8. }
```

Fatec
Diadema
Luigi Papaiz

Automação Indeterminada: While

Repetição baseada em condição (Teste no Início).



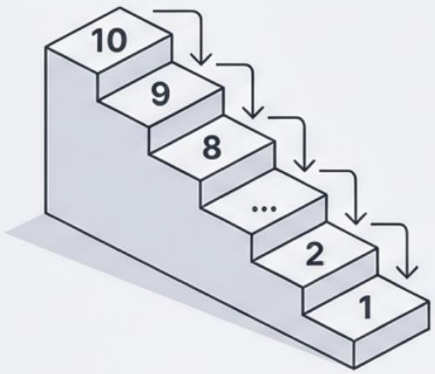

```
var opcao: Int = 0
println('Digite uma opção')
while (opcao != -1) {
    opcao = readLine().toString().toInt()
    println('Opção escolhida \$opcao')
}
println('Fim')
```

Automação Determinada: O Laço For

Iteração controlada com Intervalos.

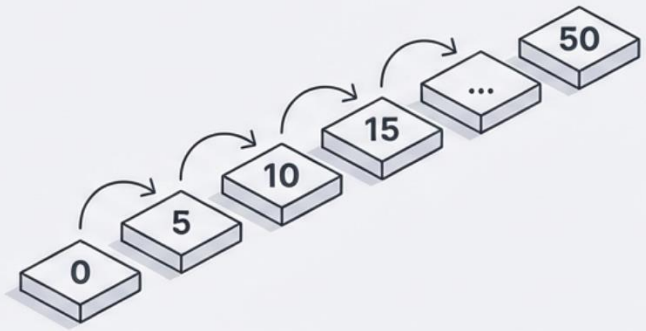
```
for (contador in 0..10) {
    println(contador)
}
```

Contagem Regressiva (downTo)




```
for (contador in 10 downTo 1) {
  ...
}
```

Passos Personalizados (step)




```
for (contador in 0..50 step 5) {
  ...
}
```

For

```
fun main() {

    for (contador in 0..10) {
        println(contador)
    }
    println("Fim do Processamento")
}
```

Fatec
Diadema
Luigi Papaiz

For – Código Fonte

```
1. fun main() {

2.     for (contador in 0..10) {
3.         println(contador)
4.     }
5.     println("Fim do Processamento")
6. }
```

Fatec
Diadema
Luigi Papaiz

For

```
fun main() {
    for (contador in 10 downTo 1) {
        println(contador)
    }
    println("Fim do Processamento")
}
```

For – Código Fonte

```
fun main() {
    for (contador in 10 downTo 1) {
        println(contador)
    }
    println("Fim do Processamento")
}
```

For

```
fun main() {
    for (contador in 0..50 step 5) {
        println(contador)
    }
    println("Fim do Processamento")
}
```

For – Código Fonte

```
1. fun main() {
2.     for (contador in 0..50 step 5) {
3.         println(contador)
4.     }
5.     println("Fim do Processamento")
6. }
```

Fatec
Diadema
Luigi Papaiz

For

```
fun main() {
    for (contador in 50 downTo 0 step 5) {
        println(contador)
    }
    println("Fim do Processamento")
}
```

For – Código Fonte

```
fun main() {
    for (contador in 50 downTo 0 step 5) {
        println(contador)
    }
    println("Fim do Processamento")
}
```

Iterando sobre Coleções (Arrays)

Acesso eficiente a valores e índices.

```
val alunos = arrayListOf('João', 'Maria', 'Marcos')

for (indice, aluno) in alunos.withIndex()) {
    println('\$indice - $aluno')
}
```

Fatec
Diadema
Luigi Papaiz

For

```
fun main() {
    val alunos = arrayListOf("João", "Maria", "Marcos")
    for ((indice, aluno) in alunos.withIndex()) {
        println("$indice - $aluno")
    }
    println("Fim do Processamento")
}
```

For - Código Fonte

```
1. fun main() {
2.     val alunos = arrayListOf("João", "Maria",
   "Marcos")
3.     for ((indice, aluno) in alunos.withIndex()) {
4.         println("$indice - $aluno")
5.     }
6.     println("Fim do Processamento")
7. }
```

Fatec
Diadema
Luigi Papaiz

For

```
fun main() {
    val alunos = arrayListOf("João", "Maria", "Marcos")
    for (i in 0..2) {
        println("$i - ${alunos[i]}")
    }
    println("Fim do Processamento")
}
```

Fatec
Diadema
Luigi Papaiz

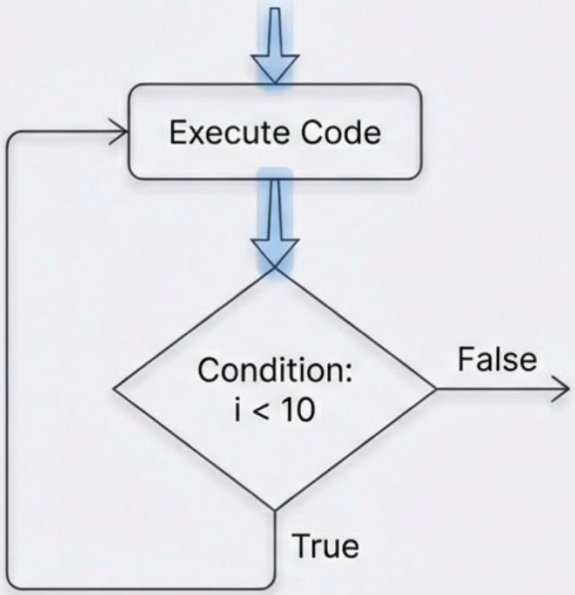
For - Código Fonte

```
1. fun main() {
2.     val alunos = arrayListOf("João", "Maria",
   "Marcos")
3.     for (i in 0..2) {
4.         println("$i - ${alunos[i]}")
5.     }
6.     println("Fim do Processamento")
7. }
```

Fatec
Diadema
Luigi Papaiz

Automação com Teste Final: Do-While

Garante execução pelo menos uma vez.




```
var i = 100
do {
    println(i)
    i++
} while (i < 10)
```

Imprime 100, depois para.

Do ... While

```
fun main() {
    var i = 100
    do{
        println(i)
        i++
    } while (i<10)
    println("Fim do Processamento")
}
```

Fatec
Diadema
Luigi Papaiz

Do ... While – Código Fonte

```
1. fun main() {
2.     var i = 100
3.     do{
4.         println(i)
5.         i++
6.     } while (i<10)
7.     println("Fim do Processamento")
8. }
```

Fatec
Diadema
Luigi Papaiz

Break (Parar)

Encerra o loop imediatamente.

```
if (i == 10) {
   break
}
```

Continue (Pular)

Pula a iteração atual e avança.

```
if (i % 2 == 1) {
   continue // Pula ímpares
}
```

Utilizando break ou continue

```
fun main() {
   for(i in 1..100)
   {
      if(i ==10){
         break
      }
      println(i)
   }
   println("Fim do Processamento")
}
```

Fatec
Diadema
Luigi Papaiz

Utilizando break ou continue- Código Fonte

```
1.  fun main() {
2.      for(i in 1..100)
3.      {
4.          if(i ==10){
5.              break
6.          }
7.          println(i)
8.      }
9.      println("Fim do Processamento")
10. }
```

Fatec Diadema Luigi Papaiz

Utilizando break ou continue

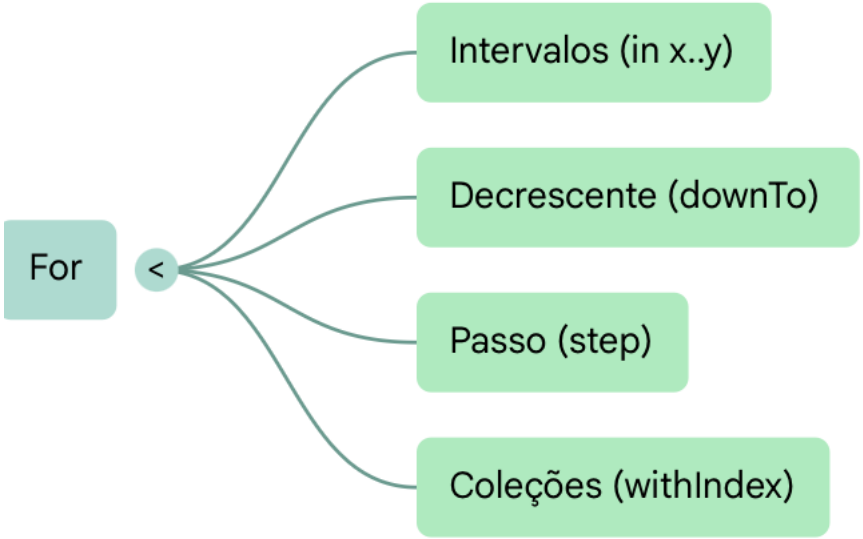
```
fun main() {
   for(i in 1..100)
   {
      if(i%2 ==1){
         continue
      }
      println(i)
   }
   println("Fim do Processamento")
}
```

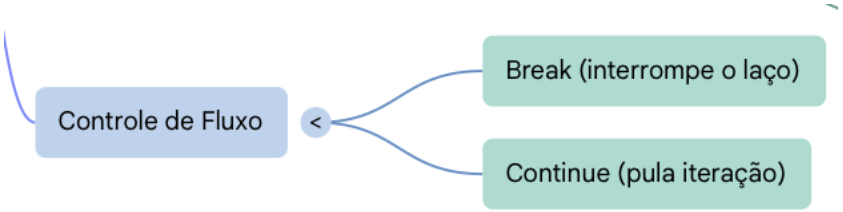
Fatec
Diadema
Luigi Papaiz

Utilizando break ou continue – Código Fonte

```
1.  fun main() {
2.      for(i in 1..100)
3.      {
4.          if(i%2 ==1){
5.              continue
6.          }
7.          println(i)
8.      }
9.      println("Fim do Processamento")
10. }
```

Fatec Diadema Luigi Papaiz


For
<
Intervalos (in x..y)
Decrescente (downTo)
Passo (step)
Coleções (withIndex)


Controle de Fluxo
<
Break (interrompe o laço)
Continue (pula iteração)

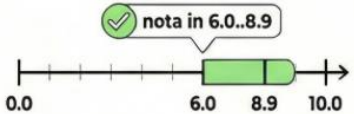
GUIA KOTLIN: ESTRUTURAS CONDICIONAIS E DE REPETIÇÃO

Contexto: Baseado na aula do Prof. Marcos Vasconcelos (Fatec Diadema).
Antes de codificar, siga 5 fases: Entender > Detalhar (E/P/S) > Fluxograma > Testes > Codificação.

ESTRUTURAS CONDICIONAIS (Tomada de Decisão)

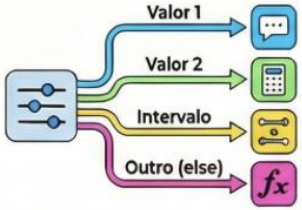
O Comando 'if' como Expressão

Em Kotlin, o 'if' pode retornar um valor diretamente para uma variável.



Operador Range (..)

Facilita a verificação de intervalos numéricos.



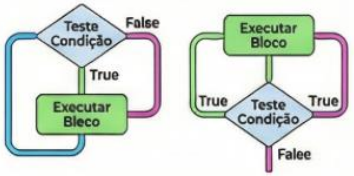
Estrutura "when"

Substitui o 'switch/case' tradicional, sendo mais legível para múltiplas condições.

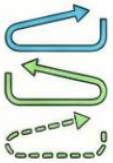
ESTRUTURAS DE REPETIÇÃO (Loops)

'while' vs. 'do-while'

O 'while' testa antes de executar; o 'do-while' executa pelo menos uma vez.



Versatilidade do 'for'



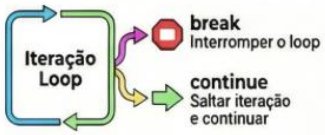
1 em 1
for (i in 0..10)

Regressivo
for (i in 10 downTo 1)

Salto 5
for (i in 0..50 step 5)

Suporta contagem progressiva, regressiva ('downTo') e saltos customizados ('step').

Controle com 'break' e 'continue'



Use 'break' para interromper o loop e 'continue' para saltar iterações.

Comparação Sintaxe de Iteração

Comando	Ação Visual	Sintaxe Exemplo
Range Padrão	Crescente (1 em 1)	`for (i in 0..10)`
downTo	Regressivo	`for (i in 10 downTo 1)`
step	Salto customizado	`for (i in 0..50 step 5)`

Ao utilizar o `if` como uma expressão em Kotlin para atribuir um valor a uma variável, qual regra deve ser seguida em blocos com múltiplas linhas?

A. Apenas valores numéricos podem ser retornados quando o `if` é usado como expressão.

B. O valor retornado será sempre a primeira expressão definida dentro do bloco escolhido.

C. A última expressão presente no bloco executado será o valor retornado para a variável.

D. É obrigatório o uso da palavra-chave `return` para extrair o valor do bloco `if`.

C. A última expressão presente no bloco executado será o valor retornado para a variável.

✓ **Isso mesmo!**

Em Kotlin, quando o `if` ou `else` são usados como expressões, o valor final do bloco selecionado é o que define o resultado da operação.

Na estrutura `when` do Kotlin, como é definida a ação padrão caso nenhuma das condições anteriores seja atendida?

A. O programa gera um erro de execução caso nenhum ramo seja correspondente.

B. Através da cláusula `default`, similar à linguagem C ou Java.

C. Utilizando o operador `->` sem nenhuma expressão à esquerda.

D. Através da cláusula `else`, que captura qualquer valor não processado pelos ramos anteriores.

D. Através da cláusula `else`, que captura qualquer valor não processado pelos ramos anteriores.

✓ **Isso mesmo!**

O `else` atua como a condição de fallback obrigatória quando o `when` é usado como expressão ou para garantir totalidade.

Qual é a principal diferença prática entre as estruturas de repetição `while` e `do-while` conforme demonstrado no material?

A. O `do-while` garante que o bloco de código seja executado pelo menos uma vez, mesmo que a condição seja falsa no início.

B. O `do-while` não permite o uso de variáveis incrementais dentro de seu bloco.

C. O `while` é utilizado apenas para contagens fixas, enquanto o `do-while` é para condições lógicas.

D. O `while` verifica a condição apenas após a execução da primeira iteração.

A. O `do-while` garante que o bloco de código seja executado pelo menos uma vez, mesmo que a condição seja falsa no início.

✓ **Isso mesmo!**

Como a verificação da condição ocorre após a execução do bloco, a primeira iteração é sempre garantida no `do-while`.

Ao utilizar o laço `for` com a sintaxe `for (contador in 0..50 step 5)`, o que o parâmetro `step` define?

A. O número total de iterações que o laço deve realizar.

B. Uma condição de interrupção imediata caso o contador atinja esse valor.

C. O valor final do intervalo, indicando onde a repetição deve parar.

D. O intervalo de incremento (passo) entre cada iteração do contador.

D. O intervalo de incremento (passo) entre cada iteração do contador.

✓ **Isso mesmo!**

O `step` modifica o incremento padrão de 1 para o valor especificado, fazendo o contador 'pular' valores.

Como deve ser estruturado um laço `for` em Kotlin para que o contador percorra valores de forma decrescente?

A. Utilizando a função `downTo`, como em `for (i in 10 downTo 1)`.

B. Aplicando um sinal negativo antes do intervalo, por exemplo `-(1..10)`.

C. Utilizando o operador de intervalo padrão invertido, como `10..1`.

D. O laço `for` em Kotlin é estritamente crescente, exigindo o uso de `while` para decréscimos.

A. Utilizando a função `downTo`, como em `for (i in 10 downTo 1)`.

✓ **Isso mesmo!**

A função `downTo` é a forma correta e idiomática de criar progressões decrescentes em Kotlin.

Considere o uso de `alunos.withIndex()` em um laço `for`. Qual é a vantagem dessa abordagem ao iterar sobre uma lista?

A. Permite acessar simultaneamente o índice da posição e o valor do elemento naquela iteração.

B. Garante que a lista seja percorrida em ordem alfabética.

C. Converte automaticamente todos os elementos da lista para o tipo `String`.

D. Reduz o consumo de memória ao evitar a criação de variáveis auxiliares.

A. Permite acessar simultaneamente o índice da posição e o valor do elemento naquela iteração.

✓ **Isso mesmo!**

O método `withIndex()` retorna pares de índice e valor, permitindo o uso de desestruturação no laço `for`.

Qual é a função da palavra-chave `break` dentro de um laço de repetição, conforme os exemplos fornecidos?

A. Interromper apenas a iteração atual e passar imediatamente para a próxima.

B. Encerrar completamente a execução do laço de repetição mais interno onde se encontra.

C. Reiniciar o laço a partir do valor inicial definido no intervalo.

D. Pausar o programa e aguardar uma entrada do usuário via teclado.

B. Encerrar completamente a execução do laço de repetição mais interno onde se encontra.

✓ **Isso mesmo!**

O `break` força a saída imediata da estrutura de loop, ignorando as condições de teste restantes.

Dada a expressão lógica `if (i % 2 == 1) continue`, qual será o comportamento de um laço que percorre de 1 a 100?

A. O programa entrará em um loop infinito pois a condição nunca será falsa.

B. O laço será encerrado assim que encontrar o primeiro número ímpar.

C. Apenas os números ímpares serão impressos no console.

D. Os números ímpares serão ignorados e apenas os números pares serão processados pelas linhas seguintes do laço.

D. Os números ímpares serão ignorados e apenas os números pares serão processados pelas linhas seguintes do laço.

✓ **Resposta correta**

O `continue` salta para a próxima iteração quando a condição (ser ímpar) é verdadeira, permitindo que apenas pares cheguem ao `println`.

Qual é o procedimento correto para ler um valor inteiro do console em Kotlin, conforme o exemplo do laço `while`?

A. Não é possível ler inteiros em Kotlin via console, apenas via interface gráfica Android.

B. Utilizar diretamente `val x: Int = readLine()`.

C. Utilizar o comando `readInt()` que converte automaticamente a entrada.

D. Utilizar a sequência `readLine().toString().toInt()`.

D. Utilizar a sequência `readLine().toString().toInt()`.

✓ **Isso mesmo!**

É necessário garantir que a entrada seja tratada como string e então convertida para o formato inteiro através de métodos de conversão.

Por que o uso do `when` pode ser preferível a múltiplas estruturas `if-else if` encadeadas?

A. O `when` oferece uma sintaxe mais limpa e legível para lidar com múltiplas ramificações de uma mesma variável.

B. O `when` é a única estrutura que permite comparar valores do tipo `Double`.

C. O `when` executa todas as condições simultaneamente, economizando tempo de processamento.

D. Não existe diferença prática ou sintática entre as duas abordagens.

A. O `when` oferece uma sintaxe mais limpa e legível para lidar com múltiplas ramificações de uma mesma variável.

✓ **Isso mesmo!**

A estrutura é projetada para ser mais concisa e organizada que longas cadeias de condicionais aninhadas.

Qual é a função do operador `indice, aluno` dentro de parênteses no laço `for ((indice, aluno) in alunos.withIndex())`?

A. Trata-se de uma desestruturação de um objeto do tipo `IndexedValue` retornado pelo iterador.

B. Define que o laço deve ser executado em duas threads diferentes.

C. Indica ao compilador para ignorar o conteúdo da lista e focar apenas nos números.

D. É uma declaração de um novo array contendo índices e nomes.

A. Trata-se de uma desestruturação de um objeto do tipo `IndexedValue` retornado pelo iterador.

✓ **Isso mesmo!**

A sintaxe permite 'desempacotar' os componentes de cada elemento iterado diretamente em variáveis distintas.

Em um laço `while (contador <= 10)`, o que acontece se a instrução `contador++` for omitida?

A. O Kotlin detecta a falha e interrompe o laço automaticamente após 100 execuções.

B. O programa entrará em um loop infinito, pois a condição de parada nunca será alcançada.

C. O laço será executado apenas uma única vez.

D. O programa gera um erro de compilação imediato.

B. O programa entrará em um loop infinito, pois a condição de parada nunca será alcançada.

✓ **Isso mesmo!**

Sem a modificação da variável de controle, a expressão lógica permanecerá verdadeira para sempre.

Se o código `val maiorNota = if (nota1 > nota2) nota1 else nota2` for executado com `nota1 = 10` e `nota2 = 10`, qual será o resultado?

A. A variável `maiorNota` ficará com valor nulo.

B. Será impresso uma mensagem de erro no console.

C. O valor atribuído será o de `nota2` (10).

D. O programa apresentará um erro por não contemplar a igualdade.

C. O valor atribuído será o de `nota2` (10).

✓ **Isso mesmo!**

Como $10 > 10$ é falso, o fluxo segue para o `else`, retornando o valor contido em `nota2`.

Sobre a estrutura `do-while`, o que define a continuidade ou término do loop?

A. O loop termina quando a condição testada ao final torna-se verdadeira.

B. O loop continua enquanto a expressão lógica avaliada no final do bloco for verdadeira.

C. O loop termina automaticamente após a décima execução para evitar sobrecarga.

D. A existência de um comando `break` é obrigatória para encerrar qualquer `do-while`.

B. O loop continua enquanto a expressão lógica avaliada no final do bloco for verdadeira.

✓ **Isso mesmo!**

Assim como no `while`, a permanência no ciclo depende da validade da condição lógica estabelecida.

Na instrução `for (i in 0..2)`, qual será a sequência exata de valores assumidos por `i`?

A. O valor será incrementado de 2 em 2, assumindo apenas 0 e 2.

B. Apenas 0 e 1.

C. 0, 1 e 2.

D. 1 e 2.

C. 0, 1 e 2.

✓ **Isso mesmo!**

Intervalos criados com `..` são fechados em ambos os lados, processando todos os inteiros do início ao fim inclusive.

Qual é a finalidade do comando `println("Opção escolhida $opcao")` conforme visto no exemplo do `while`?

A. Limpar o conteúdo do console antes de mostrar a nova opção.

B. Imprimir um texto fixo no console ignorando o valor da variável.

C. Exibir uma mensagem formatada que inclui o valor atual da variável `opcao` através de interpolação de string.

D. Realizar um cálculo matemático envolvendo a variável `opcao`.

C. Exibir uma mensagem formatada que inclui o valor atual da variável `opcao` através de interpolação de string.

✓ **Isso mesmo!**

O Kotlin usa o prefixo `$` para permitir que valores de variáveis sejam embutidos diretamente em cadeias de texto de forma simples.

Ao rodar o exemplo `for (contador in 50 downTo 0 step 5)`, qual será o segundo valor impresso pelo programa?

A. 49

B. 45

C. 40

D. 55

B. 45

✓ **Isso mesmo!**

Iniciando em 50 com um `step` de 5 em ordem decrescente (`downTo`), o próximo valor após o primeiro será $50 - 5 = 45$.

No exemplo de código do material, o que o comando `println(opcao is Int)` verifica?

A. Se a variável foi inicializada corretamente antes do laço.

B. Se o tipo de dado da variável `opcao` é compatível com o tipo Inteiro (`Int`).

C. Converte a variável para o tipo Inteiro caso ela seja uma String.

D. Se a variável `opcao` contém um valor maior que zero.

B. Se o tipo de dado da variável `opcao` é compatível com o tipo Inteiro (`Int`).

✓ **Isso mesmo!**

O operador `is` em Kotlin é utilizado para verificação de tipos (type checking) em tempo de execução.

O que acontece em um laço `do-while` se a condição de saída já for falsa no momento em que o código atinge o loop?

A. O compilador apresentará um aviso de código inalcançável.

B. O bloco de código nunca será executado.

C. O bloco de código será executado exatamente uma vez antes de sair.

D. O programa trava em uma tentativa infinita de validação.

C. O bloco de código será executado exatamente uma vez antes de sair.

✓ **Isso mesmo!**

Como a execução precede a verificação no `do-while`, o corpo do loop roda ao menos uma vez independentemente da condição.

Qual é a consequência de não fornecer o bloco `else` em uma estrutura `when` que é utilizada para atribuir valor a uma variável?

A. O programa encerra a execução com uma exceção em tempo de execução.

B. O compilador gerará um erro, exigindo que o `when` seja exaustivo.

C. A variável simplesmente mantém seu valor anterior sem sofrer alteração.

D. O Kotlin atribui automaticamente `null` se nenhuma condição for atendida.

B. O compilador gerará um erro, exigindo que o `when` seja exaustivo.

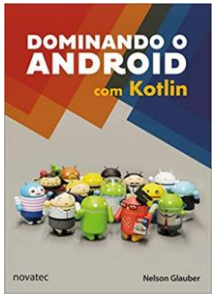
✓ **Resposta correta**

Quando usado como expressão (atribuição), o `when` deve cobrir todas as possibilidades, sendo o `else` a forma comum de garantir isso.

- Atividade 02

Referências

- GLAUBER, Nelson. Dominando o Android com Kotlin, 3ª Ed. São Paulo: Editora Novatec, 2019
- LECHETA, Ricardo R. **Android Essencial com Kotlin**. 2ª ed. São Paulo: Editora Novatec, 2018



Obrigado!

Prof. Marcos Vasconcelos de Oliveira

Fatec
Diadema
Luigi Papaiz